Valós idejű stratégiai játék

Funkcionális specifikáció és rendszerterv

Unity 6 / Windows / LAN többjátékos játék

2026. október

Tartalomjegyzék

1. A dokumentum célja **3**

2. A játék áttekintése **3**
2.1. Alapjellemzők . . . 3
2.2. A játékos célja . . . 3

3. A mérkőzés létrehozása és indítása **3**
3.1. Lobby és játékoshelyek . . . 3
3.2. Kezdőállapot . . . 4

4. Procedurális világgenerálás **4**

5. Kamera, kijelölés és egységirányítás **5**
5.1. Kamera . . . 5
5.2. Kijelölés és parancsok . . . 5

6. Egységek és szerepkörök **5**
6.1. Általános harci egység . . . 5
6.2. Bázisromboló egység . . . 5
6.3. Egységvadász . . . 5
6.4. APC . . . 6

7. Automatikus harc **6**

8. Bázisok, falak és tornyok **6**
8.1. A bázis szerkezete . . . 6
8.2. Közös báziscélpont . . . 7
8.3. A fal leomlása és a foglalás . . . 7
8.4. Bázis átadása . . . 8

9. Erőforrás-gazdálkodás **8**

10. Gyártás és fejlesztés **8**
10.1. Gyártási sor . . . 8
10.2. Fejlesztések és tornyok . . . 9

11. Sárga frakció **9**

12. Játékos kiesése és győzelem **9**

13. Hálózati architektúra **9**
13.1. A host feladatai 10
13.2. A kliensek feladatai 10

14. Mentés és visszatöltés **10**

15. Grafika és megjelenítés **10**

16. Javasolt belső programfelépítés **11**

17. Minimális elfogadási feltételek **11**

18. Kifejezetten konfigurálható vagy később pontosítandó részletek **12**

19. Fejlesztési sorrend **12**

20. Záró megjegyzés **13**

1. A dokumentum célja

Ez a dokumentum egy felülnézetes, valós idejű stratégiai játék (RTS) funkcionális specifikációját rögzíti. Célja, hogy közös referenciát adjon a játék működéséhez, a programozási feladatok felosztásához, az adatmodellek kialakításához és a későbbi teszteléshez. A játék az Alpha Wars jellegű bázisfoglalási, egységirányítási és erőforrás-gazdálkodási mechanikákból merít, de a megvalósítás saját Unity-projektben történik.

A specifikáció különválasztja a már meghozott játéktervezési döntéseket és azokat az értékeket, amelyeket később adatokkal kell beállítani. Az egységek pontos neve, életereje, sebzése, ára, gyártási ideje, hatótávolsága, mozgási költsége és az épületfejlesztések számszerű értékei nem a program szerkezetébe égetett szabályok: konfigurálható játékadatok lesznek.

2. A játék áttekintése

2.1. Alapjellemzők

- Műfaj: valós idejű, felülnézetes stratégiai játék.
- Célplatform: Windows.
- Motor: Unity 6.
- Többjátékos mód: helyi hálózaton (LAN) játszható, host–kliens modellben.
- Kamera: minden játékos saját, egymástól függetlenül mozgatható kamerával rendelkezik; a nézet tájolása rögzített, a kamera nem forgatható.
- Világ: minden résztvevő ugyanazt a játékvilágot és ugyanazt a szimulált állapotot látja, de a kameramozgás és a kezelőfelület helyi.
- Játékmód: kompetitív; a mérkőzésnek egyetlen győztese van.

2.2. A játékos célja

A játékos bázisokat foglal el, erőforrásokat termel, egységeket gyárt, fejleszti a bázisait, és harcban megsemmisíti az ellenfelek védelmét. A játékos akkor esik ki, amikor egyetlen bázis sem marad a tulajdonában. A kieső játékos minden megmaradt egysége megsemmisül. A mérkőzést az a játékos nyeri, aki az összes bázist birtokolja.

A játék nem tartalmaz kötelezően kezelt szövetségi vagy diplomáciai rendszert. A játékosok szóban megállapodhatnak, de a program nem biztosít közös gazdaságot, védelmi szövetséget vagy kötelező szövetségi szabályokat.

3. A mérkőzés létrehozása és indítása

3.1. Lobby és játékoshelyek

1. A host létrehoz egy új mérkőzést.
2. Beállítja a játékoshelyek számát, a világ méretét és a semleges sárga bázisok számát.

3. A világ mérete a játékosok számával skálázható. A sárga bázisok száma legalább a játékosok számától függő minimumot érje el, de a host ennél nagyobb értéket is választhasson.

4. A játékosok csatlakoznak a hosthoz, és elfoglalják a számukra kijelölt játékoshelyeket.

5. A mérkőzés csak akkor indulhat el, ha minden játékoshely be van töltve.

6. A világ generálása és a kezdőállapot kiosztása a host feladata, majd a host szinkronizálja a világot a kliensekkel.

A hálózati megvalósítás első változatában elegendő lehet a közvetlen LAN-csatlakozás vagy IP-cím alapú kapcsolódás. Az automatikus LAN-felderítés külön fejlesztési feladatként kezelhető.

3.2. Kezdőállapot

Minden játékos egy saját kezdőbázissal és a játék által meghatározott kezdő erőforrásokkal indul. A világban emellett semleges, sárga frakcióhoz tartozó bázisok találhatók. A sárga bázisok a mérkőzés elején nem tartoznak egyik játékoshoz sem.

A kezdő egységek, kezdő nyersanyagok, bázisok közötti minimális távolságok és a sárga bázisok számának pontos értékei konfigurálható paraméterek.

4. Procedurális világgenerálás

A világ felülnézetes, négyzetes pályaként készül. A mérete a játékosok számához és a host beállításaihoz igazodik. A generátor az alábbi elemeket helyezi el:

- játékosok kezdőbázisai;
- semleges sárga bázisok;
- terepakadályok, például hegyek és szakadékok;
- szükség esetén dekoratív, mozgást nem akadályozó elemek.

A folyók az első változatból kimaradnak. Az egységek a pálya szabad területein mozoghatnak, de nem haladhatnak át akadályokon vagy a bázisok tiltott területein. A pálya széle kemény határ: sem a kamera, sem az egységek nem hagyhatják el a világot.

A generálásnak ellenőriznie kell, hogy:

- a bázisok körül maradjon elegendő hely a kezdő egységek és az APC-k megjelenésére;
- az akadályok ne fedjék el a bázisokat és a kötelező megjelenési pontokat;
- a bázisokhoz vezető útvonalak ne legyenek véletlenül teljesen elzárva;
- az akadályok és bázisok között legyen megfelelő ütközési távolság.

Ha egy elhelyezés érvénytelen, a generátor új pozíciót keres, vagy újragenerálja az érintett részt. A pálya véletlenmagját (seed) a mentés tartalmazza, hogy a generált világ reprodukálható legyen.

5. Kamera, kijelölés és egységirányítás

5.1. Kamera

Minden kliens saját kamerát mozgat a közös világ felett. A kamera tájolása rögzített. A kamera mozgása nem hálózati játékállapot, ezért nem kell a többi játékoshoz szinkronizálni. A kamera a világ határain belül marad.

5.2. Kijelölés és parancsok

- Az egységek egérrel, téglalap alakú kijelölőkerettel csoportosan kijelölhetők.
- Jobb egérgombbal mozgási parancs adható.
- A játékos csak a saját egységeinek adhat parancsot.
- Az egységek önállóan nem járőröznek és nem üldöznek célpontot; mozgásukhoz játékosparancs kell.
- Az egységek az útvonaltervező segítségével kerülik ki a terepakadályokat.
- Az egységcsoporton belüli elrendezéshez a projektben már meglévő Python-algoritmus használható referenciaként. A végleges játékban célszerű a logikát C#-ra átültetni, hogy a futó játékhoz ne kelljen külön Python-környezet.

Az útvonaltervezéshez a pálya méretétől és az egységek számától függően rácsalapú A* vagy más, mérhető teljesítményű módszer választható. Az útvonaltervező nem dönthet tulajdonjogról és nem végezhet harci logikát; csak járható útvonalat számít.

6. Egységek és szerepkörök

A kezdeti verzió négy egységkategóriát tartalmaz: három szárazföldi harci szerepkört és egy APC-t. A három harci szerepkör mindegyikéhez 2–3 konkrét egység készülhet, de ezek közös programlogikát használnak, és adatokkal különböznek.

6.1. Általános harci egység

Közepes vagy kiegyensúlyozott sebzést okoz ellenséges egységeknek és bázisoknak. Nincs erős specializációja.

6.2. Bázisromboló egység

Nagy sebzést okoz bázisvédelmi elemeknek, és kisebb sebzést mozgó ellenséges egységeknek.

6.3. Egységvadász

Nagy sebzést okoz ellenséges egységeknek, és kisebb sebzést bázisoknak.

6.4. APC

Az APC elsődleges funkciója a lerombolt védelmű bázis elfoglalása. A bázis csak akkor foglalható el, ha a fal leomlott, vagyis minden működő védelmi torony megsemmisült. Az APC belép a bázis foglalási területére, majd a host a foglalást érvényesíti.

Minden egység adatai közé tartozhat:

- egyedi típusazonosító és megjelenítési név;
- tulajdonos játékosazonosító;
- életerő és maximális életerő;
- mozgási sebesség és támadási hatótávolság;
- támadási időköz;
- egység, torony és bázisépület elleni sebzés;
- gyártási idő és nyersanyagköltség;
- mozgáshoz szükséges olaj mennyisége;
- célpontválasztási kategória.

A konkrét számértékek konfigurációs adatok, nem a programkódba rögzített állandók.

7. Automatikus harc

Az egységek csak játékosparancsra mozognak. Ha azonban ellenséges egység kerül a támadási hatótávolságukba, az egység automatikusan tüzet nyithat rá a beállított támadási szabályok szerint. Az automatikus támadás nem indíthat el önálló üldözést: ha a célpont elhagyja a hatótávolságot, az egység nem követi automatikusan.

A játékos beállíthatja, hogy saját egységei mely játékosok egységeit támadják automatikusan. Ez a beállítás a célpontkeresést szűri, és nem jelent valódi szövetségi rendszert. A kézi támadási parancs szabályát külön kell implementálni; javasolt, hogy az explicit parancs felülírhassa az automatikus szűrést.

A sebzés kiszámítása a hoston történik. A kliens nem döntheti el önállóan, hogy egy célpont sebződött vagy megsemmisült.

8. Bázisok, falak és tornyok

8.1. A bázis szerkezete

Minden bázis tartalmaz:

- tulajdonosi állapotot (játékos, semleges sárga frakció, illetve átmenetileg gazdátlan állapot);
- falat és védelmi tornyokat;

- négy nyersanyag-termelő épületet: szén-, acél-, olaj- és betontermelőt;
- egységgyártási sort;
- egységmegjelenési és APC-foglalási területet.

A négy termelőépület kezdetben 1. szintű. A 0. szintű épület a bázison marad, de nem termel nyersanyagot. A fejlesztési maximum és a szintenkénti költségek konfigurálhatók.

8.2. Közös báziscélpont

A bázis tornyai egyetlen közös célpontot használnak. A célpontválasztás bázisszintű szolgáltatás, nem külön-külön toronylogika. Minden működő torony a közös célpontot támadja, miközben a saját tüzelési sebességét, hatótávolságát és sebzését használja.

A célpont-prioritások:

1. APC;
2. bázisromboló egységek;
3. általános harci egységek és egységvadászok.

Ha magasabb prioritású ellenség lép a bázis hatótávolságába, a bázis célpontot vált. Azonos prioritású új ellenség megjelenése nem kényszerít célpontváltást. Ha a célpont megsemmisül, vagy érvénytelenné válik, a bázis a legmagasabb prioritású elérhető kategóriából véletlenszerű célpontot választ. Ha nincs érvényes célpont, a tornyok nem tüzelnek.

8.3. A fal leomlása és a foglalás

A fal akkor omlik le, amikor az utolsó működő védelmi torony is megsemmisül. Ekkor:

- a bázis átmenetileg gazdátlan, lerombolt védelmű állapotba kerül;
- a termelőépületek szintje azonnal csökken a konfigurált rombolási szabály szerint;
- a tulajdonjog még nem változik meg;
- a bázis nem kap új védelmi tornyokat automatikusan.

A termelőépületek szintvesztésének pontos mértéke játékadat, ezért a szabály később módosítható anélkül, hogy a foglalási folyamatot át kellene írni.

Az APC csak a lerombolt falú bázist foglalhatja el. Sikeres foglaláskor:

- a bázis tulajdonosa az APC tulajdonosára változik;
- a bázis alapfalat és két kezdőtornyot kap;
- a termelőépületek a harc után megmaradt szinten működnek tovább;
- a bázis termelése és egységgyártása a tulajdonoshoz tartozik.

A sárga frakció nem foglal játékosbázisokat.

8.4. Bázis átadása

A játékosok átadhatnak bázist egymásnak. Az átadás a foglaláshoz hasonló tulajdonosváltási művelet, de nem igényli a fal lerombolását és APC belépését, hacsak a végleges szabályok másként nem rendelkeznek. Az átadás pontos kezelése – például az aktuális gyártási sor, tornyok, termelőszintek és erőforrások sorsa – konfigurálandó vagy külön szabályként rögzítendő.

9. Erőforrás-gazdálkodás

A játék négy erőforrást használ:

- **Szén:** egységgyártáshoz és bázisvédelmi fejlesztésekhez használható.
- **Acél:** egységgyártáshoz és tornyok építéséhez használható.
- **Olaj:** az egységek mozgásához szükséges.
- **Beton:** a termelőépületek fejlesztéséhez szükséges.

A pontos költség-hozzárendelés egység- és fejlesztéstípusonként konfigurálható. A nyersanyagkészlet játékoshoz tartozik, nem egyetlen bázishoz. Egy játékos minden birtokolt bázisának termelése összeadódik.

A termelés időalapú. Egy termelőépület által előállított mennyiség a szintjétől és a konfigurált termelési rátától függ. A termelés a játék szimulációs idejével számol, nem a képkockasebességgel.

Ha nincs elegendő olaj egy mozgási művelethez, a mozgás viselkedését egyértelműen definiálni kell. Javasolt alapértelmezés: a mozgási parancs megmarad, az egység megáll, majd elegendő olaj esetén folytatja a mozgást. A költség lehet távolság- és egységtípusfüggő. Ez a szabály játékadatként vagy külön mozgási szolgáltatásban kezelendő.

10. Gyártás és fejlesztés

10.1. Gyártási sor

Minden bázis saját gyártási sorral rendelkezik. A sor használata ingyenes; a sorba állított egységhez azonban nyersanyagköltség és gyártási idő tartozik.

1. A játékos egységet választ a gyártási listából.
2. A host ellenőrzi a tulajdonjogot és a szükséges nyersanyagokat.
3. Siker esetén a költséget levonja, és a gyártási tételt a sorba teszi.
4. A sor elején lévő egység a gyártási idő alapján készül.
5. A kész egység a bázis kijelölt, szabad megjelenési helyére kerül.
6. A sor a következő tétellel folytatódik.

A gyártás megszakításának visszatérítési szabálya, illetve a blokkolt megjelenési pont kezelése külön konfigurálható részlet.

10.2. Fejlesztések és tornyok

A termelőépületek beton felhasználásával fejleszthetők, és a szintjük növeli a termelési rátát. A tornyok építéséhez acél használható. A fal védelmi állapota a működő tornyokhoz kapcsolódik; az utolsó torony megsemmisülése a fal leomlását eredményezi.

A fejlesztések költségei, maximumszintjei, építési ideje és hatásai játékadatként tárolandók.

11. Sárga frakció

A sárga frakció az első változatban egyszerű, helyben maradó védelmi szerepet tölt be:

- egységei előre meghatározott helyeken állnak a bázisuk előtt;
- a közelükbe érő érvényes ellenséges célpontokat automatikusan támadják;
- nem üldözik az ellenséget, nem járőröznek, és nem mozognak önállóan;
- nem gyártanak utánpótlást.

Ez csökkenti az AI összetettségét, miközben a sárga bázisok elfoglalása továbbra is harci feladat marad. Később az utánpótlás vagy hullámok rendszere külön funkcióként hozzáadható.

12. Játékos kiesése és győzelem

A host folyamatosan ellenőrzi a játékosok bázistulajdonát.

- Ha egy játékosnak legalább egy bázisa van, tovább játszhat.
- Ha egy játékos elveszíti az utolsó bázisát, kiesik.
- A kieső játékos minden egysége megsemmisül.
- A mérkőzés akkor ér véget, amikor egy játékos az összes bázist birtokolja.
- A győztes és a mérkőzés végeredménye minden kliensen azonos.

Mivel a győzelmi feltétel egyetlen játékos összes bázis feletti tulajdonjoga, a bázisok nem tűnnek el foglaláskor, hanem tulajdonost váltanak.

13. Hálózati architektúra

A rendszer host-authoritative, vagyis a host az egyetlen hiteles forrás a közös játékállapot szempontjából.

13.1. A host feladatai

- világ generálása és mérkőzésállapot kezelése;
- játékosparancsok ellenőrzése;
- mozgás, útvonalak végrehajtása és olajfogyasztás;
- automatikus célpontválasztás és harci szimuláció;
- nyersanyagtermelés, gyártás és fejlesztések;
- bázisrombolás, foglalás, kiesés és győzelem kezelése;
- mentés és visszatöltés.

13.2. A kliensek feladatai

A kliens kezeli a kamerát, a kijelölést, a felhasználói felületet, a helyi animációkat és a játékosparancsok elküldését. A kliens megjeleníti a hosttól kapott állapotot, de nem dönthet önállóan közös játékszabályokról.

A host a parancsok végrehajtása előtt ellenőrzi többek között a játékos személyazonosságát, az egységek tulajdonjogát, a célpont érvényességét és az erőforrásokat.

14. Mentés és visszatöltés

A mentésnek tartalmaznia kell minden, a mérkőzés folytatásához szükséges állapotot:

- mérkőzésbeállítások, világméret és generálási seed;
- játékosok azonosítói, kiesési állapota és erőforrásai;
- bázisok pozíciója, tulajdonosa, falállapota, tornyai és termelőépületszintjei;
- egységek azonosítója, típusa, tulajdonosa, pozíciója és életereje;
- gyártási sorok és az aktuális gyártás előrehaladása;
- folyamatban lévő mozgási parancsok és szükséges mozgási állapot;
- játékosok automatikus célpontszűrési beállításai;
- a játékidő és a végeredményhez szükséges állapot.

A szabályos kilépéskor a host ment, majd bezárja a mérkőzést. A kliens szabályos kilépése is a mérkőzés mentését és lezárását váltja ki a projekt jelenlegi követelménye szerint. Váratlan folyamatleállás ellen időszakos automatikus mentés szükséges. A hostátvétel nem része az első változatnak.

15. Grafika és megjelenítés

A játék sprite-okból és rétegezhető képelemekből épül fel. A bázisok vizuális szintjei külön grafikai állományokkal jeleníthetők meg. A bázisokon tulajdonost jelző zászló szerepel, az

egységek felett pedig a tulajdonost azonosító felirat jelenik meg.

Az egységekhez rendelkezésre álló mozgási képkockák és animációk használhatók. A grafikák importálásakor egységesíteni kell a pixels-per-unit értéket, a pivotpontokat, az átlátszó hátteret, a sorting layereket és az ütközési méreteket. A tömeges import előtt reprezentatív egységekkel és bázisokkal próba-jelenetet kell készíteni.

A meglévő külső játékból kinyert grafikák technikai felhasználhatósága nem jelent automatikusan terjesztési vagy kereskedelmi felhasználási engedélyt. A későbbi önálló kiadáshoz saját vagy megfelelő licencű grafikákat és saját játékazonosítót kell használni.

16. Javasolt belső programfelépítés

A programot rétegekre kell bontani, hogy a játéklogika ne függjön közvetlenül a grafikai GameObjectektől.

- **Presentation/UI:** kamera, kijelölés, kezelőfelület, animációk és vizuális visszajelzés.
- **Application/Commands:** a játékosparancsok fogadása, ellenőrzése és a megfelelő szolgáltatás meghívása.
- **Domain/Simulation:** egységek, bázisok, gazdaság, harc, gyártás, célpontválasztás, kiesés és győzelem szabályai.
- **World/Pathfinding:** pályagenerálás, járhatóság, akadályok és útvonaltervezés.
- **Networking:** lobby, parancstovábbítás, állapotszinkronizáció és kapcsolatkezelés.
- **Persistence:** mentés, betöltés és mentési verziókezelés.
- **Content/Data:** egység-, épület-, költség-, sebzés- és termeléskonfigurációk.

A Domain réteg lehetőleg ne hivatkozzon Unity-megjelenítési komponensekre. A GameObjectek és MonoBehaviour komponensek a domainállapot megjelenítéséért és a Unity-események adapterezéséért feleljenek. A hálózati réteg ne tartalmazzon saját, párhuzamos játékszabályokat: a host a közös domainlogikát hívja meg.

17. Minimális elfogadási feltételek

Az első működő változat akkor tekinthető a fő funkciók szempontjából késznek, ha az alábbiak teljesülnek:

1. Egy host és legalább egy kliens ugyanahhoz a mérkőzéshez tud csatlakozni.
2. A résztvevők ugyanazokat a bázisokat, egységeket, erőforrásokat és harci eredményeket látják.
3. A kamera játékosonként külön mozgatható, és nem hagyja el a pályát.
4. Az egységek kijelölhetők, mozgási parancsot fogadnak, és nem mennek át tiltott akadályokon.
5. Az egységek a megfelelő célpontokra automatikusan tüzet nyitnak.

6. A bázis tornyai közös célpontot használnak, és a prioritási szabály szerint váltanak.
7. A tornyok megsemmisíthetők, a fal leomlik, a termelőépületek szintje csökken, majd az APC elfoglalhatja a bázist.
8. A nyersanyagtermelés, gyártási sor és épületfejlesztés működik.
9. A játékos kiesése és az egyetlen győztes meghatározása minden kliensen konzisztens.
10. A teljes mérkőzésállapot menthető és visszatölthető.
11. A generált világban nincsenek elérhetetlen, hibásan elhelyezett vagy egymást átfedő kritikus objektumok.

18. Kifejezetten konfigurálható vagy később pontosítandó részletek

Az alábbiak nem akadályozzák a rendszertervezést, mert adatként vagy elkülönített szabályként kezelhetők:

- egységek pontos nevei, statisztikái, költségei és gyártási ideje;
- a termelőépületek termelési rátái, maximális szintjei és fejlesztési költségei;
- a bázisromboláskor elveszített termelőszintek pontos mértéke;
- a mozgás olajköltségének képlete és az olajhiány pontos kezelése;
- a gyártás lemondásának visszatérítése és a blokkolt megjelenési pont kezelése;
- a bázisátadás részletes szabályai;
- a kézi támadási parancs és az automatikus célpontszűrés pontos prioritása;
- a világméret, a bázisszám és az akadályok konkrét generálási paraméterei;
- a hálózati frissítési gyakoriság, az automatikus LAN-felderítés és a mentési időköz.

19. Fejlesztési sorrend

A hálózati kockázat korai felmérése érdekében a host–kliens prototípust az első fejlesztési szakaszban kell elkészíteni.

1. Projektstruktúra, adatok és egyszerű tesztpálya.
2. Kamera, kijelölés, egységmozgás és útvonaltervezés.
3. Host–kliens kapcsolat és minimális állapotszinkronizálás.
4. Bázisok, tornyok, közös célpontválasztás és harc.
5. APC-foglalás, gazdaság, gyártás és fejlesztések.
6. Procedurális világgenerálás és sárga frakció.

7. Lobby, játékosbeállítások, kiesés és győzelem.

8. Mentés/betöltés, kezelőfelület, hibajavítás és többjátékos tesztelés.

20. Záró megjegyzés

Ez a specifikáció a játék jelenleg meghatározott funkcionális alapját rögzíti. A pontos számértékek és egyensúlyozási paraméterek külön adatfájlokba vagy Unity ScriptableObject konfigurációkba kerüljenek. A legfontosabb architekturális szabály, hogy a játékszabályok, a hálózati továbbítás, a mentés és a grafikai megjelenítés ne keveredjenek egyetlen nagy vezérlőosztályba.